

## Comunicación de aplicaciones y MongoDB

Cuando una aplicación, o un script, necesita almacenar, consultar o manipular datos en MongoDB, debe primero generar un canal de comunicación para indicar las operaciones que debe realizar. En el proceso se define cómo viaja la información, quién la interpreta y qué formato se utiliza.

### Modelo de comunicación

La comunicación entre una aplicación y una instancia de MongoDB sigue el clásico modelo de Cliente-Servidor. En este modelo la aplicación se comporta como cliente y mongo es quien recibe y responde a la solicitud actuando como servidor.

Ya establecida la conexión, la aplicación debe:

- Autenticarse y mantener una conexión persistente con la instancia de MongoDB
- Construir una solicitud con la información de la operación a realizar
- Preparar los datos en el formato aceptado por el driver de comunicación
- Esperar la respuesta del servidor con los datos solicitados o el resultado de la operación

### Driver como intermediario

En este contexto, un driver es un componente de software o librería específica para un lenguaje de programación, que actúa como “traductor” entre la aplicación y la instancia de MongoDB. Su rol es abstraer la complejidad de la conexión y la comunicación con el servidor de MongoDB. Para ello, permite llevar a cabo las siguientes operaciones de manera muy sencilla:

- Establecer la conexión: Gestiona la conexión y la autenticación
- Serialización/Deserialización: Convierte las estructuras de datos nativas del lenguaje de programación (como objetos o diccionarios) al formato de comunicación de MongoDB (BSON) y viceversa
- Ejecutar comandos: Permite a la aplicación enviar comandos de consulta, inserción, actualización, etc.

### Estructura de la cadena de conexión

A la hora de establecer una conexión, el driver requiere que le indiquemos la instancia a la cuál se tiene que conectar, la base de datos particular a la cual ejecutar las operaciones y también la información de autenticación. Esto se indica en una cadena de texto con alguno de los siguientes formatos:

- Para conexiones locales o de servidores simples

```
"mongodb://usuario:contraseña@host:puerto/"
```

- Para conexiones gestionadas por MongoDB Atlas o clusters con registro DNS SRV

```
"mongodb+srv://usuario:contraseña@host/"
```

Ambos formatos admiten autenticación en la cadena de texto pero también existen otros protocolos y formas de autenticación.

## Alternativas a la hora de conectar una aplicación

### Driver nativo de MongoDB

Esta alternativa nos permite trabajar de manera más cercana a la comunicación real con el motor de bases de datos, utilizando únicamente la abstracción mínima que provee el driver para gestionar la conexión, la serialización de datos y el intercambio de mensajes con el servidor. En este enfoque, el modelado, la validación y la estructura de los datos quedan bajo responsabilidad del desarrollador, sin incorporar capas adicionales como las que ofrece un ODM.

### Object Document Modeling (ODM)

Un ODM es una componente de la aplicación que actúa como una capa de abstracción que vincula estructuras propias del lenguaje, como objetos, a documentos o colecciones presentes en el motor de bases de datos. De esta manera, no tenemos que preocuparnos de ciertas complejidades que pueden presentarse al usar un driver nativo cuando las operaciones se vuelven más complejas. Permite definir esquemas, que modelan las colecciones de documentos, permite realizar validaciones en la aplicación y muchas otras capacidades interesantes para manipular los datos. Es el equivalente a un ORM (Object Relational Mapping) de las bases de datos relacionales, pero para bases de datos basadas en documentos, como MongoDB.